

CODE ABAP2

2주차 멘토링

2025. 12. 23 (화)



CL3 이창근

전역 변수와 지역 변수는 언제 사용해야하는가?

- 기본 원칙은 결국 **"필요한 범위에서만 변수가 살아있게 하라"** - 변수의 생명주기를 최소화하는 것이 핵심
- 지역변수를 사용해야 하는 경우
 - 특정 로직 내에서만 필요한 임시 데이터
 - FORM, FUNCTION, METHOD 내부에서만 쓰이는 계산용 변수(lv_tabix 같은 루프 카운터 등)
 - 같은 이름으로 다른 목적의 변수가 필요할 때

```
* DATA: gv_count TYPE i,  
FORM process_orders USING pv_tables.  
  " 지역변수: 이 서브루틴에서만 사용  
  DATA lv_count TYPE i.  
  
  lv_count = lines( pv_tables ).  
ENDFORM.
```

전역 변수와 지역 변수는 언제 사용해야하는가?

▪ 전역변수를 사용해야 하는 경우

- 여러 서브루틴/메소드에서 공유해야 하는 데이터
 - 프로그램 전체에서 참조하는 변수 (ALV에서 사용하는 gv_layout 등)
 - **Selection screen 파라미터 (pa_, so_)**
- Internal table, Workarea 등 여러 곳에서 사용할 때
- 성능상 이유로 무조건 재사용해야하는 테이블
 - **대용량 테이블을 반복적으로 생성/소멸하면 시간(오버헤드)이 너무 많이 들음**

* 전역: ALV 관련 객체와 출력 데이터

```
DATA: go_alv TYPE REF TO cl_gui_alv_grid,  
      go_container TYPE REF TO cl_gui_custom_container,  
      gt_output TYPE TABLE OF ty_output,  
      gt_fieldcat TYPE lvc_t_fcat,  
      gt_input TYPE TABLE OF ty_input. “엔트리가 100만개라면?
```

아직도 잘 모르겠어요...

▪ 그럼 일단 지역 변수부터 써보세요

- 해보고 좀 쓰다보면 전역 변수로 빼야한다 싶으면 그 때 빼도 되고, 애당초 바로 몸으로 느끼게 됩니다.
- 지역변수로 썼더니 성능 문제가 생기거나, 다른 곳에서도 써야하는데 지역 변수로만 썼다던가..
- 데이터가 엄청 많은 Internal Table을 지역 변수로 처리해보면 성능 문제가 바로 느껴짐
 - 엔트리가 10만 개 정도 되는 테이블이면 불러오는 데 못해도 4초는 걸림
 - 그 테이블을 사용하는 서브루틴이나 Function Module을 **10번 불러오면...? 프로그램 실행에 40초...?**
 - **전역변수로 쓰면 10번 불러와도 4초면 되는데요?**

"의심스러우면 지역변수로 시작" - 나중에 필요하면 전역으로 올리면 됨

네이밍 컨벤션 활용 - `lv_` (local variable), `gv_` (global variable) 은 접두사로 꼭 구분하세요

디버깅 용이성 - 지역변수를 잘 써먹었다면 디버그할 때 변수 추적이 더 쉽습니다

절차 지향 프로그래밍(Procedural Programming)

- **절차를 따라 차례대로 한줄씩 내려가는 프로그래밍**

- 절차 지향 프로그래밍은 마치 요리할 때 레시피를 따르는 것과 비슷함.
- 재료를 준비하고, 재료를 다듬고, 조리하는 단계를 차례대로 따라감.
 - 절차 지향 프로그래밍에서는 프로그램을 여러 개의 순서대로 진행되는 절차(함수나 명령)들로 구성.
 - 이런 방식은 프로그램이 어떻게 실행되어야 하는지, 단계별로 무엇을 해야 하는지를 중시

예를 들어, "학교에서 집까지 가는 길을 찾는 프로그램"을 만든다고 가정하면,
절차 지향 방식으로 접근하면, 먼저 학교에서 출발해야 한다는 절차를 정의하면서 시작.
그 다음 1. 길을 따라 가는 방법, 2. 교통 수단을 선택하는 방법 3. 어느 곳에서 내리는 지 등
차례로 명령으로 만들어 나가는 방식.

클래스와 객체를 배우기 전까지 만든 ABAP Report Program은 모두 절차지향 프로그래밍이었음

객체 지향 프로그래밍(Object-Oriented Programming)

▪ 좀 더 현실 세계에 가까운 방식으로 프로그래밍하는 것

- 현실 세계에는 많은 '객체'들이 있듯이, OOP에서는 모든 것을 객체로 바라본다
- 여기서 **객체**란 데이터(속성)와 그 데이터와 관련된 기능(메서드)을 하나로 묶은 것을 의미

객체 지향 방식을 적용할 때는 프로그램이 해결해야 할 문제를 작은 부분들로 나누는게 먼저.

- 그 후 각 부분을 '객체'로 만들면, 이 객체들은 서로 '상호작용'하면서 전체 프로그램이 작동하게 됩니다.
- 객체들은 각자의 데이터를 가지고 있으며, 다른 객체들과 메시지를 주고받으며 상호작용할 수 있음.

아까 이야기한 "학교에서 집까지 가는 길을 찾는 프로그램"을 만든다고 가정하면,

객체 지향 방식으로 접근하면, '학교', '집', '길', '교통 수단' 등을 각각의 객체로 정의하는 것으로 해결한다.

이 객체들이 서로 상호작용하면서 최종 목적지인 집에 도착하는 방법을 만드는 것.

사실상 대부분의 객체 지향 프로그래밍은 절차 지향 프로그래밍에 객체의 개념을 얹은 것과 다를 바가 없음

아직도 잘 모르겠어요... 클래스? 객체? 인스턴스?

- 원래 어려운게 맞습니다

- 이제 막 알파벳과 기초 영문법을 뚝 학생에게 논리적인 작문을 가르치는 것과 비슷합니다
- 붕어빵 기계 - 슈붕, 팔붕 / 사람 - 김연아, 이창근 / 펭귄 - 뽀로로, 핑구
- 같은 붕어빵, 사람, 펭귄이더라도 가진 것(속성), 할 줄 아는 것(메서드)는 다를 수 있다고 생각하면 이해가 쉬움.
- 클래스는 설계도, 객체는 그 설계도로 만든 변수(Type이 클래스), 객체가 실제로 저장공간에 올라가면 인스턴스
- 객체 = 인스턴스는 사실상 같은 것이라고 봐도 무방함

어려워도 해보자! 1. 캡슐화 (Encapsulation)

▪ 실생활 비유 : 자동차 운전

- 운전자(사용자)는 핸들, 페달, 기어만 조작
- 엔진 내부(프로그램 내부)가 어떻게 작동하는지는 몰라도 됨 + 엔진 내부는 기업 기밀이라 노출되면 안됨
- 내부 구조를 숨기고 사용자에게는 필요한 것만 노출하는 것이 캡슐화
 - 속성과 메서드를 클래스 내부에 숨기고(은닉화) 접근을 제어하며 통제한다.
 - PUBLIC, PROTECTED, PRIVATE 섹션으로 가시성 제어를 제어

```
CLASS lcl_example DEFINITION.  
  PRIVATE SECTION. "여기 선언된 변수는 외부에서 직접 접근 불가  
    DATA: mv_value TYPE i.  
  PUBLIC SECTION. "여기 선언된 변수는 외부에서 접근 가능  
    METHODS: get_value RETURNING VALUE(rv_result) TYPE i,  
              set_value IMPORTING iv_value TYPE i.  
ENDCLASS.
```

어려워도 해보자! 2. 상속 (Inheritance) 🧑🧒

▪ 실생활 비유 : 가족 관계

- 자식은 부모의 특징을 물려받음
- 부모가 "키가 크다"면 자식도 그 특징 보유
- 자식만의 고유한 특징도 추가 가능 (잘 생겼음, 똑똑함)
 - 부모 클래스의 속성과 메서드를 자식 클래스가 물려받음 (캡슐화도 유지 가능)
- INHERITING FROM 구문을 사용해 구현, 코드 재사용성이 향상되고 계층 구조 표현 가능

```
CLASS lcl_child DEFINITION INHERITING FROM lcl_parent.  
  "부모의 PUBLIC, PROTECTED 멤버 사용 가능 (PRIVATE 직접접근 안됨)  
  "PRIVATE 멤버에 접근하려면 부모의 메서드를 이용해 간접접근 가능  
  "ex)엄마 금고에 돈을 써먹으려면 엄마한테 돈 꺼내달라고 요청해야 함  
ENDCLASS.
```

어려워도 해보자! 3. 추상화 (Abstraction) 🎨

- 실생활 비유 : TV 리모컨

- "전원", "채널", "볼륨" 같은 핵심 기능(공통 메소드)만 정의
- 삼성 TV든 LG TV든 같은 버튼으로 조작, 구체적인 구현(추상 메소드)을 어떻게 할지는 개발자가 알아서
- 공통 메소드는 정의만 하고, 추상 메소드는 상속받은 자식 클래스가 대신 구현한다
 - 즉 추상화는 상속에 의해 구현이 가능한 것 (상속이 먼저, 추상화가 나중에)
 - ABSTRACT 클래스와 REDEFINITION을 사용하여 구현 가능
 - 추상화를 하면 설계와 구현을 별개로 분리하여, 유지보수에 유연한 구조를 만들 수 있다.

" 추상 클래스 - 설계도

```
CLASS lcl_payment DEFINITION ABSTRACT.
```

```
    PUBLIC SECTION.
```

```
        METHODS: process_payment ABSTRACT " 구현 없이 정의만
```

```
                IMPORTING iv_amount TYPE p,
```

```
                show_receipt. " 공통 메소드 (공통 부분은 공간 부족으로 생략)
```

```
ENDCLASS.
```

" 구체적인 구현 1 - 신용카드

```
CLASS lcl_credit_card DEFINITION INHERITING FROM lcl_payment.
```

```
    PUBLIC SECTION.
```

```
        METHODS: process_payment REDEFINITION. " 재정의
```

```
ENDCLASS.
```

```
CLASS lcl_credit_card IMPLEMENTATION.
```

```
    METHOD process_payment.
```

```
        WRITE: / '신용카드로', iv_amount, '원 결제 처리'.
```

```
        show_receipt( ).
```

```
    ENDMETHOD.
```

```
ENDCLASS.
```

어려워도 해보자! 4. 다형성 (Polymorphism) 🎭

- 실생활 비유 : "울음소리 내기"

- 개는 "멍멍", 고양이는 "야옹", 소는 "음메" 각자 울음소리를 다르게 낼 수 있음

- 같은 동물이고, 같은 "울음소리 내기" 명령이지만 동물마다 다르게 동작(메소드)를 구현 할 수 있음

- 하나의 같은 인터페이스를 사용해서, 여러 가지로 구현이 가능함

- 인터페이스와 REDEFINITION을 사용하여 구현 가능

- 유지보수에 유연한 코드를 만들 수 있고, 추후에 기능을 확장하기가 용이함

" 인터페이스 정의

```
INTERFACE lif_animal.  
    METHODS: make_sound,  
             move.  
ENDINTERFACE.
```

" 개 클래스

```
CLASS lcl_dog DEFINITION.  
    PUBLIC SECTION.  
        INTERFACES: lif_animal.  
ENDCLASS.
```

```
CLASS lcl_dog IMPLEMENTATION.  
    METHOD lif_animal~make_sound.  
        WRITE: / '멍멍! 🐶'.  
    ENDMETHOD.
```

```
    METHOD lif_animal~move.  
        WRITE: / '네 발로 달립니다'.  
    ENDMETHOD.  
ENDCLASS.
```

" 새 클래스

```
CLASS lcl_bird DEFINITION.  
    PUBLIC SECTION.  
        INTERFACES: lif_animal.  
ENDCLASS.
```

```
CLASS lcl_bird IMPLEMENTATION.  
    METHOD lif_animal~make_sound.  
        WRITE: / '짹짹! 🐦'.  
    ENDMETHOD.
```

```
    METHOD lif_animal~move.  
        WRITE: / '날아갑니다'.  
    ENDMETHOD.  
ENDCLASS.
```

어려워도 해보자! 4. 다형성 (Polymorphism)

- 오버로딩 (Overloading) : 메소드 중복, **같은 메소드 이름**, **다른 매개변수** (같은 클래스 내)
 - "전화해(엄마)" → 연락처에서 찾아서 전화 / "전화해(010-1234-5678)" → 특정 번호로 전화
 - 동일한 이름의 기능을 매개변수와 반환되는 변수의 TYPE을 다르게 사용해 여러 개로 파생시킬 때 사용
 - ABAP은 오버로딩을 지원하지 않으므로, 비슷한 효과를 내는 것 밖에 방법이 없음
- 오버라이딩 (Overriding) : 메소드 재정의, **같은 메소드 이름**, **같은 매개변수** (부모-자식 관계)
 - 엄마: "청소해" → 걸레질 + 먼지털이 / 딸: "청소해" → 로봇청소기 돌림
 - 자식 클래스에서 부모 클래스의 기능을 다시 정의하고 싶을 때 사용하면 좋음
 - 부모 클래스의 메소드를 자식이 재정의하여 코드의 재사용성을 높임

어려워도 해보자! 4. 다형성 (Polymorphism) 🎭

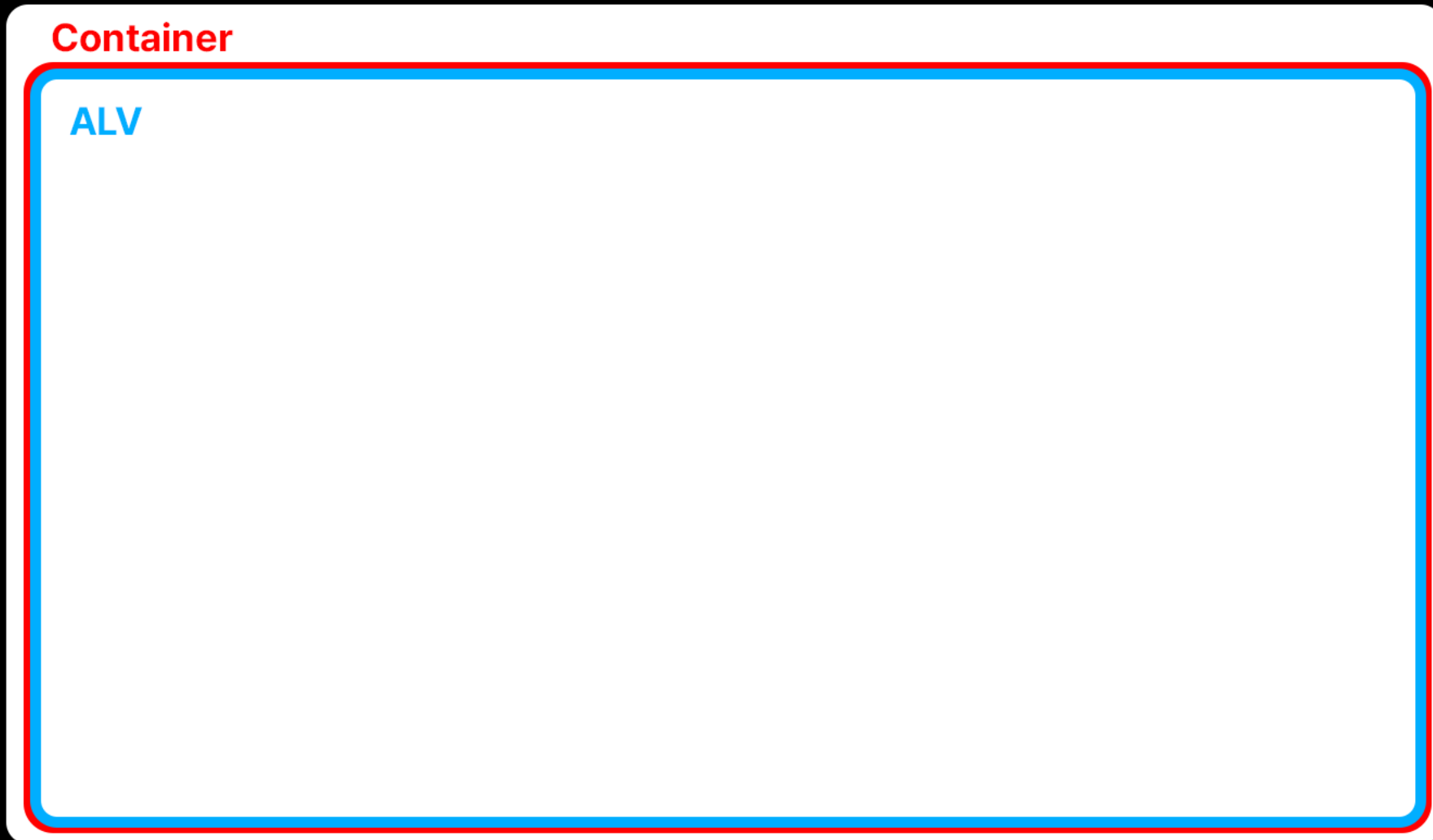
구분	Interface 다형성	오버로딩	오버라이딩 다형성
정의	동일한 인터페이스를 여러 클래스가 구현	같은 이름, 다른 매개변수의 메서드	자식 클래스가 부모 메서드를 재정의
ABAP 지원	✅ 완전 지원	❌ 미지원	✅ 완전 지원
키워드	INTERFACE , IMPLEMENTATION	-	REDEFINITION , INHERITS FROM
관계	클래스 간 독립적 (구현 관계만)	-	상속 관계 필수
사용 시점	컴파일 타임에 타입 결정, 런타임에 동작 결정	컴파일 타임 결정	컴파일 타임에 타입 결정, 런타임에 동작 결정
장점	느슨한 결합, 다중 인터페이스 가능	메서드 이름 재사용	기존 코드 재사용, 확장 용이
대안	-	OPTIONAL 파라미터, 다른 메서드명 사용	-
예시	서로 다른 동물 클래스가 같은 인터페이스 구현	(지원 안 됨)	자식 클래스가 부모의 display() 재정의

ALV (ABAP List Viewer)

- SAP에서 데이터를 표 형태로 표시하는 표준 UI 컴포넌트(구성요소)
 - 정렬, 필터링, 합계 등 다양한 기능을 자동으로 제공
- SCREEN과 Container가 ALV보다 먼저 선행되어야 하는 이유
 - SCREEN (화면): ALV가 표시될 물리적 공간(윈도우)을 제공
 - Container (컨테이너) : SCREEN 내에서 ALV 객체가 위치할 영역을 정의하는 역할
- 이 2개가 ALV를 출력하려면 왜 필요한가?
 - ALV는 GUI 컨트롤 객체로, 화면에 직접 배치될 수 없음
 - Container가 SCREEN과 ALV 사이의 중간 다리 역할을 함
 - Container가 화면의 특정 영역을 ALV에게 할당해줌 (Layout 버튼을 눌러 스크린 페인터를 실행한 이유)

ALV (ABAP List Viewer)

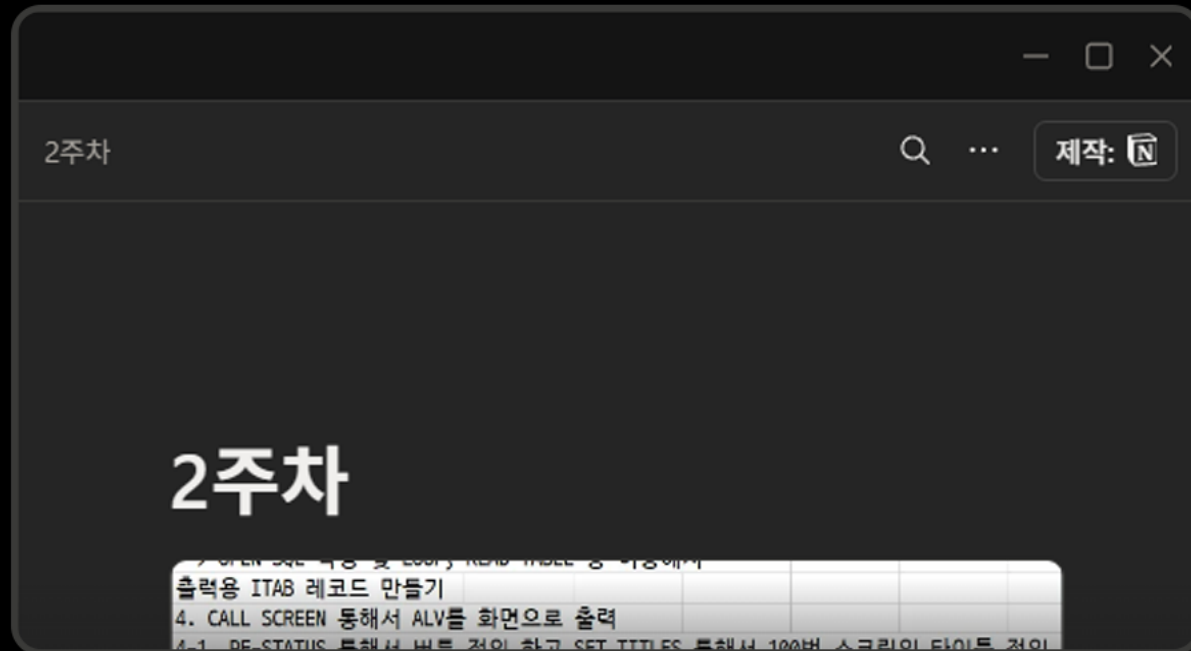
SCREEN



ALV (ABAP List Viewer) 작업 순서

- 첨부한 노션 링크를 참고하여 작업하기

https://www.notion.so/2-2d151b10cdcd801393feeea5682b875a?source=copy_link



ALV (ABAP List Viewer) 실습 문제

- SELECTION-SCREEN을 사진과 같이 만듭니다. (기본값 지정, 파라미터는 읽기 전용 처리, 해당 조건으로 SQL문)
- **gs_book-smoker**의 값이 'X'이면 **icon** 필드에 **icon_activate** 값을 세팅
- **gs_book-invoice**의 값이 'X'이면 'A'로 변경
- ALV를 다음 사진과 같이 만듭니다
 - icon 필드와 sbook의 PK들은 PK로 파랗게 표시
 - 줄무늬 표시 처리, 컬럼 가로폭 자동지정, 다중 행 선택모드, ALV 제목 표시

Condition	
Airline	LH
Connection Number	400

Single Flight Booking

↳	Status	항공사 코드	항공기 연결 번호	출발 날짜	예약 번호	고객 유형	흡연자 여부	수하물 무게	무게 단위	인보이스 상태
		LH	400	2025.05.11	78	P		0.0000	KG	
		LH	400	2025.05.11	98	B		0.0000	KG	
		LH	400	2025.05.11	113	P		0.0000	KG	
		LH	400	2025.05.11	119	P		0.0000	KG	A
		LH	400	2025.05.11	170	P		0.0000	KG	
		LH	400	2025.05.11	171	P		0.0000	KG	
		LH	400	2025.05.11	182	P		0.0000	KG	A
		LH	400	2025.05.11	185	B		0.0000	KG	
		LH	400	2025.05.11	190	P		0.0000	KG	
		LH	400	2025.05.11	211	P		0.0000	KG	A
		LH	400	2025.05.11	224	P		0.0000	KG	
		LH	400	2025.05.11	240	P		0.0000	KG	
		LH	400	2025.05.11	252	P		0.0000	KG	A

필드 이름	참조 필드
carrid	sbook-carrid
connid	sbook-connid
fldate	sbook-fldate
bookid	sbook-bookid
custtype	sbook-custtype
smoker	sbook-smoker
luggweight	sbook-luggweight
wunit	sbook-wunit
invoice	sbook-invoice

ALV (ABAP List Viewer) 실습문제 추가조건

- 필드 카탈로그 서브루틴을 만들 때, 오늘 실습 방식처럼 매개변수로 받아 필드를 APPEND 할 수 있도록 해주세요.
- Disvariant Type의 Workarea를 선언하여 Layout을 저장할 수 있도록 하세요.
- 필드 카탈로그에서 luggweight를 APPEND 할 때, 단위 참조 필드(cfieldname)을 지정할 수 있도록 해주세요.
- 아이콘 필드는 DDIC 참조할 시 이름이 빈칸이 되버리므로, ICON일 시에는 coltext를 '상태' 라고 바꿔주세요.